

Object-Oriented Programming Using Java

Inheritance and Polymorphism

Contents

1. Thừa kế là gì
2. Từ khóa protected
3. Constructor của lớp con
4. Overriding methods và Hiding methods
5. Phương thức `java.lang.Object.toString()`
6. Từ khóa final và super

References:

<https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>

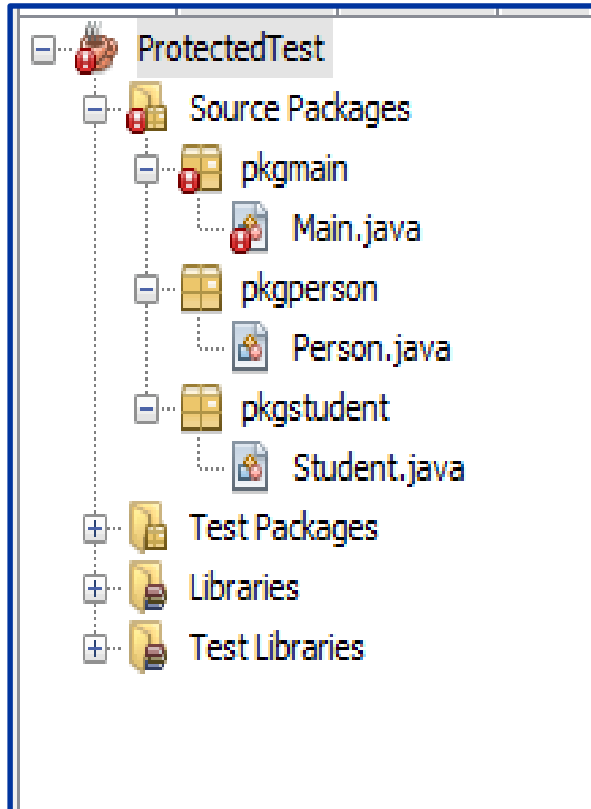
<https://docs.oracle.com/javase/tutorial/java/landl/polymorphism.html>

Ví dụ 01

protected

- Biến và phương thức khai báo **protected** được truy cập
 - Trong cùng 1 lớp
 - Trong cùng 1 package
 - Từ các lớp con (subclass)

Ví dụ 02



```
package pkgperson;

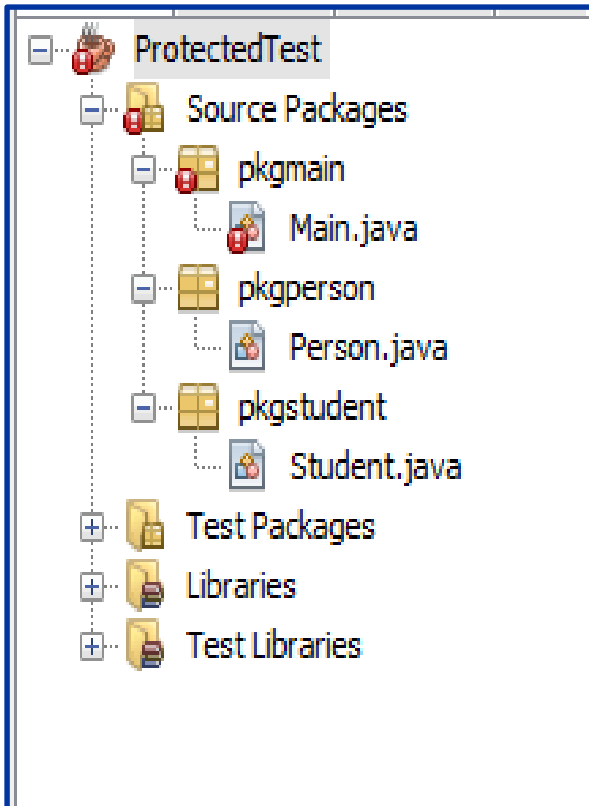
public class Person {
    protected String name, address;
    protected void input() {
        //input name and address from user
    }
}
```

```
package pkgstudent;

import pkgperson.Person;

public class Student extends Person {
    int id;
    public void inputID() {
        //input id from user
    }
    public void output() {
        System.out.println(id+"-"+name+"-"+address);
    }
}
```

Ví dụ 02



```
6 package pkgmain;
7
8 import pkgstudent.Student;
9
10 public class Main {
11     public static void main(String[] args) {
12         Student stud = new Student();
13
14         stud.input();
15         stud.inputID();
16         stud.output();
17     }
18 }
19
20 *
```

input() has protected access in Person

(Alt-Enter shows hints)

- protected -
lớp Main
không kế
thừa Student

public

Subclass constructor

- Khi tạo đối tượng của lớp con thì đối tượng của lớp cha **tự động được tạo trước** cho nên:
 1. Constructor của lớp con phải chỉ định constructor của lớp cha
 2. Nếu không chỉ định thì constructor không tham số của lớp cha sẽ được gọi

Ví dụ 03 – có chỉ định

```
class Person{
    protected String name, address;
    public Person(String name) {
        this.name = name;
    }
    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }
}

class Student extends Person{
    private int id;
    public Student(int id, String name) {
        super(name);
        this.id = id;
    }
    public Student(int id, String name, String address) {
        super(name, address);
        this.id = id;
    }
}
```

*Có chỉ định
constructor
của lớp cha*

Ví dụ 04 – không chỉ định

```
24  class Person{
25      protected String name, address;
26      public Person() {
27      }
28      public Person(String name, String address) {
29          this.name = name;
30          this.address = address;
31      }
32  }
33  class Student extends Person{
34      private int id;
35      public Student(int id, String name) {
36          this.id = id;
37      }
38      public Student(int id, String name, String address) {
39          super(name, address);
40          this.id = id;
41      }
42  }
```

*KHÔNG chỉ định
constructor của
lớp cha*

Ví dụ 05 - lỗi vì...

```
9  class Person {
10      String name, address;
11
12      public Person(String name, String address) {
13          this.name = name;
14          this.address = address;
15      }
16  }
17  public class Student extends Person
18  {
19      int id;
20      public Student(int id, String name, String address) {
21          this.name = name;
22          super.address = address;
23          this.id = id;
24      }
25  }
```

Test 1

```
6 package pkgtest;
7 class A {
8     protected int x, y;
9     public A(int xx, int yy) {
10         x = xx; y = yy;
11     }
12 }
13 public class B extends A
14 {
15     int z;
16     public B() {
17         System.out.println("Hello");
18     }
19     public static void main(String[] args) {
20         B b = new B();
21     }
22 }
```

a) Hello

b) Compile-time error

c) Run-time error

d) Nothing

`new B()` → gọi constructor 0 tham số của lớp A → lỗi

Test 2

```
7 package pkgtest;
8 class A {
9     protected int x, y;
10    public A(int xx, int yy) {
11        x = xx; y = yy;
12    }
13 }
14 public class B extends A
15 {
16     int z;
17     public B(int zz) {
18         z=zz;
19     }
20 }
```

Lỗi tại dòng 17

Test 3

```
7 package pkgtest;
8 class A {
9     protected int x, y;
10    public A(int xx, int yy) {
11        x = xx; y = yy;
12    }
13 }
14 public class B extends A
15 {
16     int z;
17    public B(int zz) {
18        super(10, 20);
19        z=zz;
20        System.out.println(x+y+z);
21    }
22    public static void main(String[] args) {
23        new B(30);
24    }
25 }
```

a) 0

b) 30

c) 60

d) Compile-time error

e) Run-time error

Test 4

```
7 package pkgtest;
8 class A {
9     protected int x, y;
10    public A(int xx, int yy) {
11        x = xx; y = yy;
12    }
13 }
14 public class B extends A
15 {
16     int z;
17     public B(int zz) {
18         z=zz;
19         super(10, 20);
20         System.out.println(x+y+z);
21     }
22     public static void main(String[] args) {
23         new B(30);
24     }
25 }
```

call to super must be first statement in constructor

(Alt-Enter shows hints)

Test 5

```
7  package pkgtest;
8  class A {
9      protected int x, y;
10     public A(int xx, int yy) {
11         x = xx; y = yy;
12     }
13 }
14 public class B extends A
15 {
16     int z;
17     public B(int zz) {
18         super(zz, zz);
19         z=zz;
20         System.out.println(x+y+z);
21     }
22     public static void main(String[] args) {
23         new B(30);
24     }
25 }
```

a) 30

b) 60

c) 90

d) Compile-time error

e) Run-time error

Test 6

```
7 package pkgtest;
8
9 class A {
10     protected int x, y;
11     public A() {
12         System.out.println("Hello A!");
13     }
14     public A(int xx, int yy) {
15         x = xx; y = yy;
16     }
17 }
18
19 public class B extends A
20 {
21     int z;
22     public B(int zz) {
23         z=zz;
24         System.out.println("Hello B!");
25     }
26     public static void main(String[] args) {
27         new B(30);
28     }
29 }
```

a) 30

b) Hello B!

c) Hello A! Hello B!

d) Hello B! Hello A!

e) Compile-time error

Overriding Methods

```
class Person{  
    String name, address;  
    void output(){  
        System.out.print(name + "-" + address);  
    }  
}
```

```
class Student extends Person{  
    int id;  
    void output(){  
        System.out.print(id + "-");  
        super.output();  
    }  
}
```

Phương thức của lớp con
định nghĩa lại phương
thức của lớp cha, *có cùng
tên, cùng danh sách tham
số và kiểu trả về*

@Override annotation

```
class Person {  
    public void output() {  
    }  
}  
class Student extends Person{  
    @Override  
    public void output() {  
    }  
}
```

Ví dụ 06 - @Override

```
33 class Person{
34     protected String name, address;
35     public Person(String name, String address) {
36         this.name = name;
37         this.address = address;
38     }
39     @Override
40     public String toString(){
41         return String.format("%s-%s", name, address);
42     }
43 }
44 class Student extends Person{
45     private int id;
46     public Student(int id, String name, String address) {
47         super(name, address);
48         this.id = id;
49     }
50     @Override
51     public String toString(){
52         return String.format("%s-%s", id, super.toString());
53     }
54 }
```

Ví dụ 06 – java.lang.Object

```
29 // mọi lớp (đều ngầm hiểu) kế thừa từ lớp java.lang.Object
```

```
30 @class Person extends Object{
31     protected String name, address;
32     public Person(String name, String address) {
33         this.name = name;
34         this.address = address;
35     }
36     @Override
37     public String toString(){
38         return String.format("%s-%s", name, address);
39     }
40 }
41 }
```

Override hàm
toString() của
lớp Object

```
42
43 class Student extends Person{
44     private int id;
45     public Student(int id, String name, String address) {
46         super(name, address);
47         this.id = id;
48     }
49     @Override
50     public String toString(){
51         return String.format("%s-%s", id, super.toString());
52     }
53 }
```

Override hàm
toString() của
lớp Person

Ví dụ 06 – Kết quả

```
17 public static void main(String[] args) {  
18     Person p1 = new Person("Person01", "Address01");  
19     System.out.println(p1.toString()); // --> toString() của lớp Person  
20  
21     Student p2 = new Student(2192100, "Student02", "Address02");  
22     System.out.println(p2.toString()); // --> toString() của lớp Student  
23  
24     Person p3 = new Student(2192101, "Person-Student??", "Address03");  
25     System.out.println(p3.toString()); // --> toString() của lớp Student  
26 }
```

Overriding methods: gọi
phương thức của lớp con
(Student)

toString() được gọi mặc định

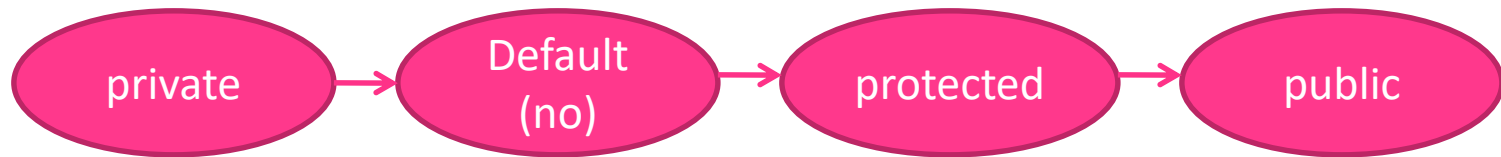
```
17 public static void main(String[] args) {  
18     Person p1 = new Person("Person01", "Address01");  
19     System.out.println(p1.toString()); // --> toString() của lớp Person  
20  
21     Student p2 = new Student(2192100, "Student02", "Address02");  
22     System.out.println(p2.toString()); // --> toString() của lớp Student  
23  
24     Person p3 = new Student(2192101, "Person-Student??", "Address03");  
25     System.out.println(p3.toString()); // --> toString() của lớp Student  
26 }
```

Không cần
gọi hàm
toString()

```
27 System.out.println(p1); // --> gọi toString() của lớp Person  
28 System.out.println(p2); // --> gọi toString() của lớp Student  
29 System.out.println(p3); // --> gọi toString() của lớp Student
```

Modifiers của phương thức @Override

- Modifiers được sắp xếp theo thứ tự tăng dần về phạm vi truy xuất (từ nhỏ đến lớn)



- Phương thức @Override của lớp con phải định nghĩa modifier **lớn hơn hoặc bằng** lớp cha

```
36  class Person extends Object{
37
38      public void output() {
39      }
40
41      protected void input() {
42      }
43  }
```

```
57  class Student extends Person{
58      @Override
59      protected void output() {
60      }
61
62      @Override
63      public void input() {
64      }
65  }
```

Polymorphism – Đa hình

```
17 public static void main(String[] args) {  
18     Object obj1 = new Object();  
19     Object obj2 = new Person("Tom", "HSU");  
20     Object obj3 = new Student(100, "Jerry", "HSU");  
21     System.out.println(obj1); //obj1 invokes toString() implicitly  
22     System.out.println(obj2); //obj2 invokes toString() implicitly  
23     System.out.println(obj3.toString()); //obj3 invokes toString() Explicitly  
24 }
```

Output:

java.lang.Object@15db9742

Tom-HSU

100-Jerry-HSU

Ví dụ 07 – Đa hình

```
18 public static void main(String[] args) {  
19     //khai báo mảng  
20     Object arr[]=new Object[5];  
21  
22     Object o1=new Object();  
23     Object o2=new Person("Tran Cong Nhat Phuong",1993);  
24     Object o3=new Student("2170317","Tran Cong Nhat Phuong",1993);  
25  
26     //khởi tạo mảng  
27     arr[0]=o1;  
28     arr[1]=o2;  
29     arr[2]=o3;  
30     arr[3]=new Person("Nguyen Huu Vinh",1995);  
31     arr[4]=new Student("2130317","Nguyen Tan Toan",1994);  
32  
33     //xuất mảng  
34     for(int i=0;i<arr.length;i++){  
35         //arr[i].output() bị lỗi vì lớp Object không có hàm output()  
36         System.out.println(arr[i].toString());  
37     }  
38 }
```

Ví dụ 08 - ArrayList

```
19  public static void main(String[] args) {  
20      ArrayList<Student> list = new ArrayList<Student>();  
21      list.add(new Student(2171101, "Student 01"));  
22      list.add(new Student(2171102, "Student 02"));  
23      list.add(new Student(2171103, "Student 03"));  
24  
25      // Cách 1  
26      for (int i = 0; i < list.size(); i++) {  
27          list.get(i).output();  
28      }  
29  
30      // Cách 2  
31      for (Student student : list) {  
32          student.output();  
33      }  
34  }
```

Hiding Methods

```
class Person{  
    public void output(){  
    }  
    public static void input() {  
    }  
}  
  
class Student extends Person{  
    @Override  
    public void output(){  
    }  
  
    // Hiding  
    public static void input() {  
    }  
}
```

Lớp cha và lớp con
định nghĩa phương
thức tĩnh cùng chữ
ký và kiểu trả về

Ví dụ 09 - Hiding Methods

```
27 class Person {
28     String name, address;
29     public Person(String name, String address) {
30         this.name = name;
31         this.address = address;
32     }
33     static void output(){
34         System.out.println("The static method in Person");
35     }
36
37 class Student extends Person{
38     int id;
39     public Student(int id, String name, String address) {
40         super(name, address);
41         this.id = id;
42     }
43     static void output(){
44         System.out.println("The static method in Student");
45     }
46 }
```

run:

The static method in Person
The static method in Person

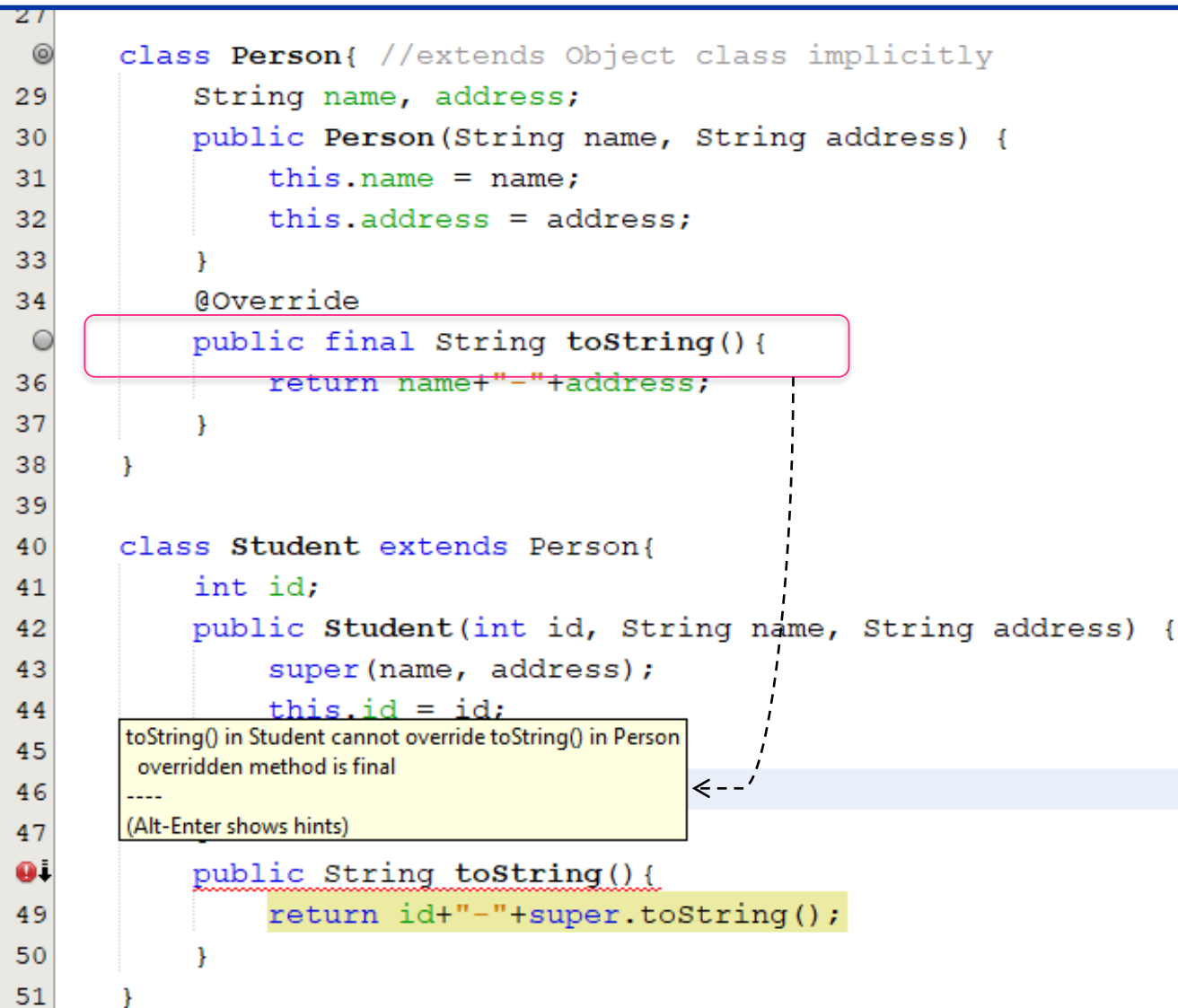
```
Person person1 = new Person("Tom", "HSU");
Person person2 = new Student(100, "Jerry", "HSU");
person1.output();
person2.output();
```

final keyword

- *final* method:
 - Lớp con **không được phép** override phương thức final của lớp cha
- *final* class:
 - Lớp final KHÔNG CÓ lớp con
 - KHÔNG ĐƯỢC PHÉP kế thừa từ lớp final
- *final* field (biến) là một **hằng số** (constant)
`final public double PI = 3.14;`

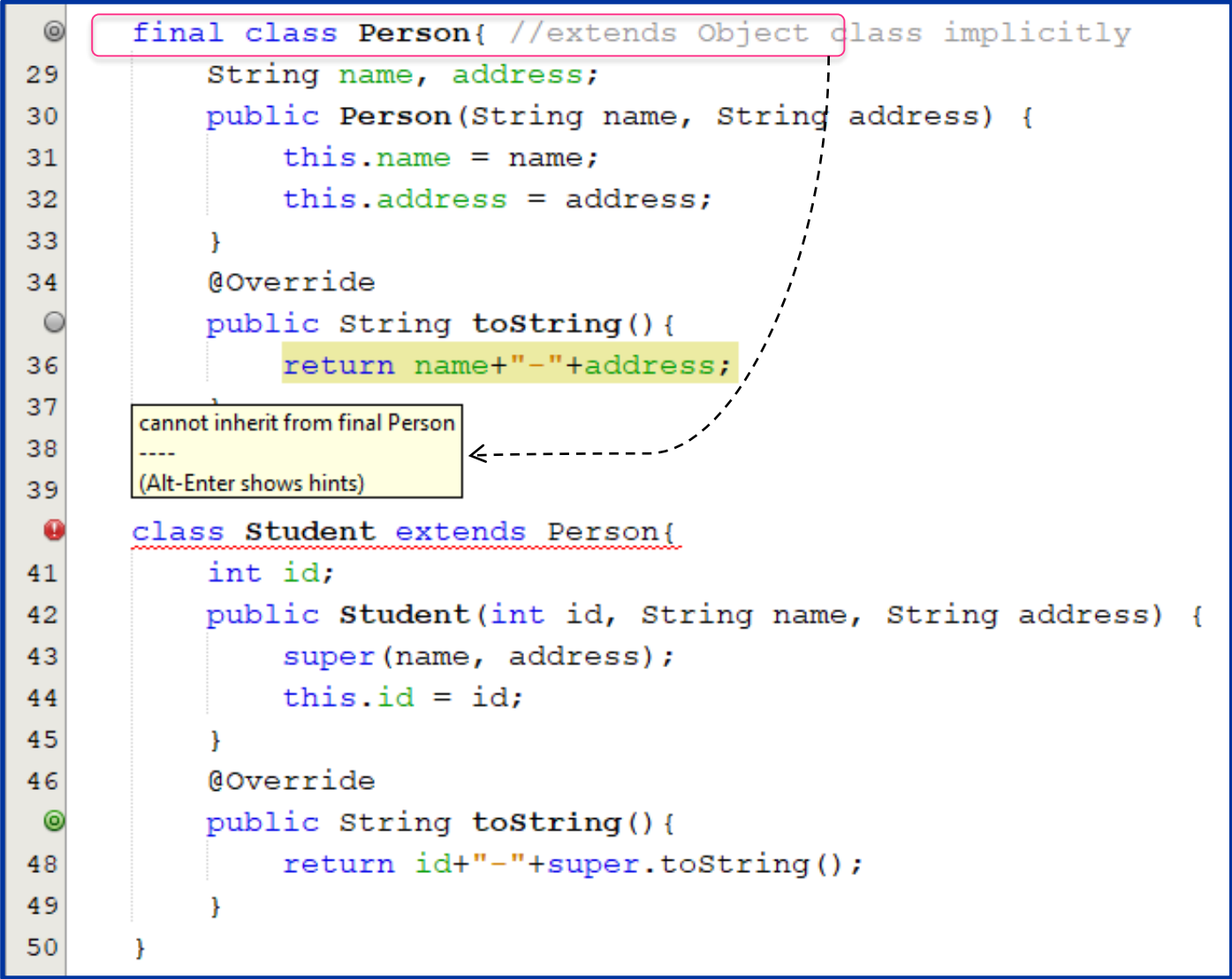
final Methods

```
27
28 ③ class Person{ //extends Object class implicitly
29     String name, address;
30     public Person(String name, String address) {
31         this.name = name;
32         this.address = address;
33     }
34     @Override
35     public final String toString(){
36         return name+"-"+address;
37     }
38 }
39
40 class Student extends Person{
41     int id;
42     public Student(int id, String name, String address) {
43         super(name, address);
44         this.id = id;
45
46         toString() in Student cannot override toString() in Person
47         overridden method is final
48         ----
49         (Alt-Enter shows hints)
50     public String toString(){
51         return id+"-"+super.toString();
52     }
53 }
```



final Class

```
29  final class Person{ //extends Object class implicitly
30      String name, address;
31      public Person(String name, String address) {
32          this.name = name;
33          this.address = address;
34      }
35      @Override
36      public String toString(){
37          return name+"-"+address;
38      }
39
40  cannot inherit from final Person
41  ----
42  (Alt-Enter shows hints)
43
44  class Student extends Person{
45      int id;
46      public Student(int id, String name, String address) {
47          super(name, address);
48          this.id = id;
49      }
50      @Override
51      public String toString(){
52          return id+"-"+super.toString();
53      }
54  }
```



The screenshot shows an IDE window with a Java code editor. The code defines a `final class Person` and a `class Student extends Person`. A red error icon is present next to the `class Student extends Person` line. A yellow tooltip box displays the error message: "cannot inherit from final Person", followed by "----" and "(Alt-Enter shows hints)". A dashed line points from the tooltip to the `final class Person` declaration. The code for `Person` includes fields `name` and `address`, a constructor, and a `toString` method. The `Student` class has an `id` field, a constructor that calls `super`, and a `toString` method that calls `super.toString()`.

super keyword

```
13 public class A {
14     protected int n=100;
15     public void hello() {
16         System.out.println("Hello A!");
17     }
18     public static void main(String[] args) {
19         B b = new B();
20         b.helloWorld();
21         b.outputN(300);
22     }
23 }
24 class B extends A{
25     private int n=200;
26     @Override
27     public void hello() {
28         System.out.println("Hello B!");
29     }
30     public void helloWorld() {
31         hello();
32         this.hello();
33         super.hello();
34         System.out.println("Hello World!");
35     }
36     public void outputN(int n) {
37         System.out.println(n);
38         System.out.println(this.n);
39         System.out.println(super.n);
40     }
41 }
```